Experiment 6: Serial Adder/Subtractor using Flip-Flops

Digital Systems and Microcontrollers, IIIT-H

Monsoon'26

1 Introduction

A 4-bit *parallel* adder uses four full adders, and the carry ripples from one to the next through wires, all in the same instant. In this experiment you will instead work **serially**: one bit per clock, starting from the least significant bit, using a single 1-bit circuit and **one flip-flop** that remembers the carry from one clock to the next.

You will first build a serial adder, and then extend it so that a select line chooses between addition and subtraction (Section 3).

- Build the circuit first in Tinkercad, then on the test kit.
- **The Tinkercad simulation must be done individually.**
- Every gate is a basic CD40xx gate (NAND, NOR, XOR). The flip-flops are built from NAND gates as shown in Section 2.

Reading the circuit diagrams: inputs carrying the same label (for example every A_i) are connected to the same switch. A wire that jumps over another wire is *not* connected to it; connections are marked with a dot.

2 Flip-Flops from NAND Gates

Both flip-flops below are *clocked* (level-triggered): they respond while CLK is high and hold their state while CLK is low. G3 and G4 form the cross-coupled NAND latch from Experiment 5.

Each flip-flop has an active-low clear input CLR' on G4. While $\mathrm{CLR}' = 0$ (with CLK low), Q is forced to 0 without any clock. In this experiment, CLR' is driven from a RESET switch through a NAND gate used as an inverter, so RESET = 1 clears the flip-flop.

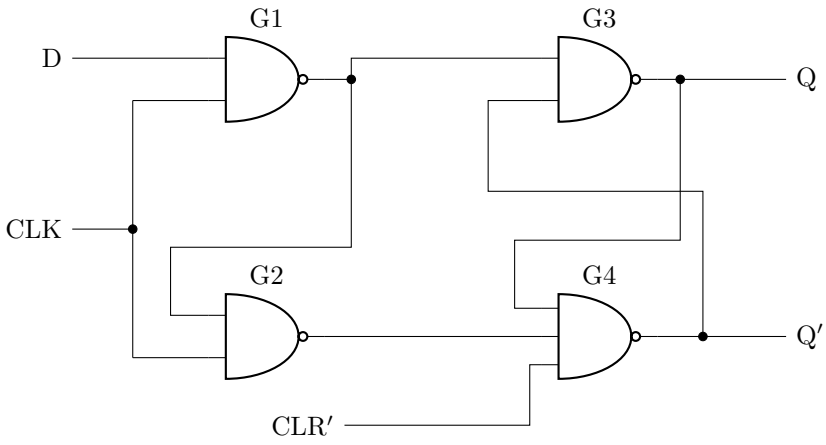


Figure 1: Clocked D flip-flop with clear. G1, G2, G3: 2-input NAND. G4: 3-input NAND. While CLK = 1, Q follows D; while CLK = 0, Q holds.

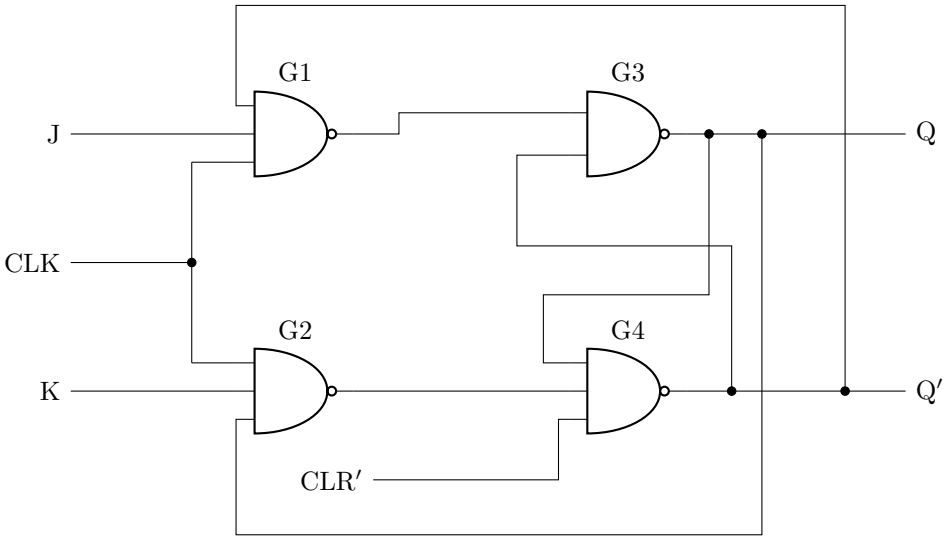


Figure 2: Clocked JK flip-flop with clear. G1, G2, G4: 3-input NAND. G3: 2-input NAND. On a clock pulse: J = 1, K = 0 sets Q = 1; J = 0, K = 1 clears Q = 0; J = K = 0 holds Q.

Tinkercad: as in Experiment 5, the cross-coupled gates may stall the simulation at start-up. Start every simulation with RESET = 1; this gives the latch its initial nudge. Then set RESET = 0.

3 Serial Adder/Subtractor

3.1 How a serial adder works

A serial adder has two blocks: a 1-bit **full adder** and a **D flip-flop** (Figure 1) that holds the carry.

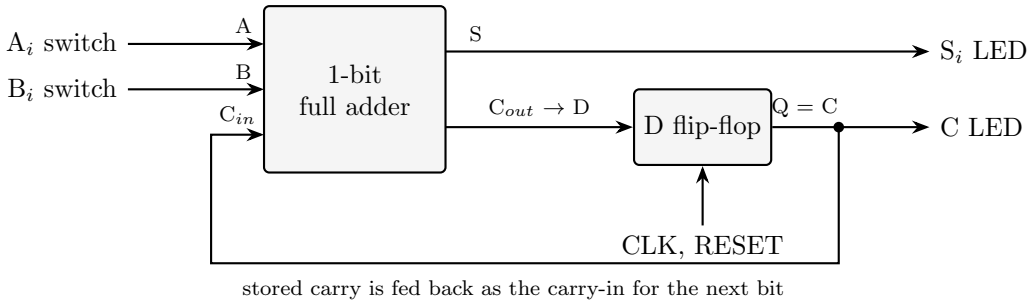


Figure 3: Serial adder block diagram.

On each clock, the full adder adds the current bits A_i, B_i and the stored carry C:

- The sum bit S_i appears on its LED immediately.
- C_{out} goes to the D input, but is *not* stored until CLK is pressed.
- When CLK is pressed, the flip-flop stores C_{out}. Its output C becomes the carry-in for the next bit.

After 4 clocks, all four sum bits have been produced, and the flip-flop holds the final carry, which is the 5th bit of the sum.

The full adder must behave as follows:

A_i	B_i	C	S_i	C_{out}
0	0	0	0	0
0	0	1	1	0
0	1	0	1	0
0	1	1	0	1
1	0	0	1	0
1	0	1	0	1
1	1	0	0	1
1	1	1	1	1

Table 1: Full adder truth table.

Example: 0110 + 0111 (6 + 7 = 13).

Clock	Bit	A_i	B_i	C before CLK	S_i	C after CLK
1	0	0	1	0	1	0
2	1	1	1	0	0	1
3	2	1	1	1	1	1
4	3	0	0	1	1	0

$S_3S_2S_1S_0 = 1101 = 13$, final carry 0.

Task 1: Build the full adder from XOR and NAND gates, connect it to the D flip-flop of Figure 1 as in Figure 3, and add a RESET inverter. Verify it with the procedure below.

3.2 Procedure

1. Set RESET = 1, then RESET = 0. The C LED is now 0.
2. Set A_0 and B_0 on the switches. Read S_0 from the LED and note it down.
3. Press CLK once. The carry from bit 0 is now stored in C.
4. Repeat steps 2 and 3 for bits 1, 2 and 3, always reading S_i *before* pressing CLK.
5. After the 4th clock, the C LED shows the final carry.

3.3 Adding a select line

Task 2: Modify your serial adder by adding one more input switch, the select line **SEL**:

- **SEL = 1:** the circuit adds, A + B, exactly as before.
- **SEL = 0:** the circuit subtracts, A − B.

Everything else stays the same:

- The bits are entered one per clock, LSB first, and the output bit is read on the same LED before each clock.
- One flip-flop stores the value passed from one bit to the next (a carry when adding; a borrow or carry when subtracting).
- RESET clears the circuit before every operation. SEL is set before RESET and is not changed during the 4 clocks.

When subtracting, the four output bits must equal A − B in 4-bit two's complement, and the flip-flop's value after the 4th clock must tell you whether A < B. In your report, state how to read the sign from that value.

You may use NAND, NOR and XOR gates and either flip-flop from Section 2; you may modify a flip-flop if your design needs it. Try to keep the number of extra gates small.

3.4 Hints

There are two common ways to approach this. Pick one.

- **Borrow.** Fill in the table below for 1-bit subtraction $A_i - B_i - Bw_{in}$, and compare it with Table 1. The difference bit and the sum bit are the same function. How is the borrow-out related to C_{out}? What single change to one input of the carry logic turns one into the other?
- **Two's complement.** $A - B = A + \overline{B} + 1$. How can B_i be inverted only when SEL = 0? Where can the "+1" come from in a serial circuit, where bit 0 is added in the first clock?

A_i	B_i	Bw_{in}	D_i	Bw_{out}
0	0	0		
0	0	1		
0	1	0		
0	1	1		
1	0	0		
1	0	1		
1	1	0		
1	1	1		

Table 2: 1-bit subtraction: D_i is the difference bit, Bw_{out} the borrow-out.

3.5 Checking your design

Use these operations to test your circuit. The expected output bits are the same whichever approach you chose.

Operation	SEL	Expected output bits (bit 3 to bit 0)	Decimal
0110 + 0111	1	1101	$6 + 7 = 13$
1010 − 0011	0	0111	$10 - 3 = 7$
0011 − 1010	0	1001	$3 - 10 = -7$

Table 3: Test operations.